

Chapitre 1 : Introduction

Dans ce cours nous allons découvrir le format JSON. Dans le monde numérique d'aujourd'hui, un format de données particulièrement apprécié pour sa simplicité et son efficacité permet de structurer et d'échanger des informations de manière fluide entre diverses applications et systèmes. Souvent utilisé pour les API et les services web, il facilite la communication entre serveurs et clients. Les développeurs apprécient ce format pour sa compatibilité avec de nombreux langages de programmation, et son adoption croissante en fait un outil incontournable dans le développement web moderne. Son côté minimaliste et intuitif le rend également très lisible, ce qui en fait un choix populaire pour les échanges de données.

1.1 Qu'est-ce que JSON ?

JavaScript Object Notation (JSON) est un format de données textuel dérivé de la notation des objets du langage JavaScript. Il concurrence XML pour la représentation et la transmission d'information structurée.

Créé par Douglas Crockford entre 2002 et 2005, la première norme du JSON est ECMA-404 d'Ecma International qui a été publiée en octobre 2003. Il est également décrit en 2017 par la RFC 8259 de l'Internet Engineering Task Force qui se veut compatible avec Ecma-404 et ECMA-404.

Des bibliothèques pour le format JSON existent dans de nombreux langages de programmation.

« WIKIPEDIA »

1.2 Caractéristique

Un texte en JSON comprend :

- deux types composés :
 - des objets JavaScript, ou ensembles de paires « nom » (ou « clé ») / « valeur »,
 - des listes ordonnées de valeurs (tableau) ;
- quatre types scalaires :
 - des booléens : prend la valeur *true* ou *false*,
 - des nombres : un nombre décimal signé qui peut contenir une part fractionnable ou élevée à la puissance. Le JSON n'admet pas les nombres inexistants (NaN), et ne distingue pas les entiers et les flottants,
 - des chaînes de caractères : une séquence de 0 ou plus caractères Unicode. À l'instar des clés, elles sont obligatoirement entourées de guillemets,
 - la valeur null : qualifie l'absence de valeur.

Chacun de ces types peut être utilisé pour constituer un document.

1.3 Exemples JSON vs XML vs YAML

Voici un exemple au format JSON :

```
{
  "menu": {
    "id": "file",
    "value": "File",
    "popup": {
      "menuitem": [
        { "value": "New", "onclick": "CreateNewDoc()" },
        { "value": "Open", "onclick": "OpenDoc()" },
        { "value": "Close", "onclick": "CloseDoc()" }
      ]
    }
  }
}
```

Equivalent au format XML :

```
<menu id="file" value="File">
  <popup>
    <menuitem value="New" onclick="CreateNewDoc()" />
    <menuitem value="Open" onclick="OpenDoc()" />
    <menuitem value="Close" onclick="CloseDoc()" />
  </popup>
</menu>
```

Equivalent au format YAML :

```
menu:
  id: file
  value: File
  popup:
    menuitem:
      - value: New
        onclick: CreateNewDoc()
      - value: Open
        onclick: OpenDoc()
      - value: Close
        onclick: CloseDoc()
```

Chapitre 2 : Les types en JavaScript Object Notation

Json est donc le diminutif de JavaScript Object Notation. C'est un format qui va permettre de représenter des données et qui va être très utilisé dans le web pour échanger des informations avec des services externes. On l'a dit, vous pourrez communiquer avec une API pour recevoir ou envoyer des informations.

Prenons un exemple de JSON qui va vous parler, puisque je vais vous afficher le JSON de 2 utilisateurs de notre base de données coursmysql :

```

{} okjson > ...
1  [
2      {
3          "id_user": "1",
4          "firstname": "James",
5          "lastname": "Bond",
6          "gender": "M",
7          "date_of_birth": "2007-07-07",
8          "city": "Londres",
9          "weight_kg": "87"
10     },
11     {
12         "id_user": "2",
13         "firstname": "Jack",
14         "lastname": "Bauer",
15         "gender": "M",
16         "date_of_birth": "2004-12-24",
17         "city": "New-York",
18         "weight_kg": "124"
19     }
20 ]

```

Vous avez déjà fait du JavaScript, donc ça doit tous vous sembler familier. On va un peu parler des types. Le site référence pour JSON est <https://www.json.org/json-en.html> .

2.1 Les types simple

2.1.1 Les nombres

Les nombres vont être représentés comme une série de chiffres. On peut avoir des nombres négatifs, à virgules, ou encore exposant etc.

```

[
    {
        "nb1": 10,
        "nb2": -64,
        "nb3": 42.98,
        "nb4": 2e10
    }
]

```

2.1.2 Les chaînes de caractères

Comme dans la majorité des langages de programmations, les chaînes de caractères vont être délimitées par des doubles guillemets. C'est le seul caractère qui va être fonctionnel dans le cadre de JSON. On a comme en PHP et dans d'autres langages, la possibilité de mettre le backslash pour utiliser par exemple un saut de ligne ou encore pour afficher le guillemet double, etc. :

```
[
  {
    "string1": "Hello World !!",
    "string2": "Welcome to the world of \"JSON\"",
    "string3": "On peut sauter une ligne \ncomme en PHP"
  }
]
```

Dans l'exemple que j'ai donné de notre base de données, vous avez dû remarquer que tous les champs étaient en chaîne de caractère. Ce sera bien entendu le format le plus utilisé.

2.1.3 Les booléens

On a ensuite les booléens que vous connaissez bien, qui donne comme valeur soit vraie soit fausse :

```
[
  {
    "bool1": true,
    "bool2": false
  }
]
```

2.1.3 Le Null

On a le type Null, qui permet de marquer une absence de valeur :

```
[
  {
    "valeur": null
  }
]
```

Tous ces types simples vous pouvez les associer bien entendu :

```
[
  {
    "nom": "Dunia",
    "prenom": "Julien",
    "age": 26,
    "tiktok": null
  }
]
```

On va maintenant voir le deuxième type : les types complexes.

2.2 Les types complexe

2.2.1 Les Tableaux

Vous le savez aussi, ils permettent de représenter une liste de valeurs :

```
{
  "utilisateurs": [
    {
      "id": 1,
      "nom": "Bond",
      "email": "jbond@cfitech.be"
    },
    {
      "id": 2,
      "nom": "Bauer",
      "email": "jbauer@cfitech.be"
    }
  ]
}
```

On sait bien entendu utiliser des tableaux dans des tableaux etc.

2.2.1 Les Objets

Le type objet fonctionne de manière un peu plus complexe. On commence par ouvrir une accolade, ensuite on mettra une chaîne de caractère puis deux points suivis d'une valeur. On mettra des virgules à chaque fois pour séparer les valeurs :

```
{
  "nom": "Dunia",
  "prenom": "Julien",
  "age": 26
}
```

Détaillons un peu plus. Après les accolades, nous avons à chaque fois une clé entre guillemet suivie de sa valeur séparée par 2 points. Les clés valeurs sont séparées par des virgules.

Je vous ai un peu déjà entraîné à ce que vous le voyez...Le fait de mettre dans tous mes exemples précédents les accolades en faisait des objets.

En effet ça permet de représenter des données structurées. Dans l'exemple ci-dessus, je représente une personne qui est caractérisée par un nom, un prénom et un âge. J'aurais pu mettre le tout sur une même ligne, mais pour la visibilité je préfère vous le montrer comme ça.

On peut avoir des JSON avec juste un type simple sans accolade.

On peut bien entendu avoir une valeur qui est de n'importe quel type dont tableau ou objet :

```
{
  "nom": "Dunia",
  "prenom": "Julien",
  "age": 26,
  "metier": {
    "nom": "Developpeur Web",
    "langageMaitrise": ["JAVA", "PHP"],
    "anneeXp": 10,
    "certifie": true
  }
}
```

Avec ce dernier type, nous avons vu tous les types existant pour le JSON.

Pour créer un json, il suffit d'aller dans VS code et de faire nouveau fichier, puis vous rajouter l'extension « .json » à la fin du nom. Créez un fichier « **demoTypes.json** ».